

Build a YouTube Downloader

Single videos and whole playlists with Python and yt-dlp

A practical, step-by-step Python project

Start with an empty folder. Finish with a small program that saves one video, or a whole playlist in a folder with the playlist's name, using the exact YouTube titles.

Alireza Khajehvandi

 Portfolio •  GitHub •  LinkedIn
 YouTube •  Telegram •  Instagram

How to use this tutorial

This is a hands-on guide for people who are new to Python. Complete the chapters in order and stop at every *checkpoint*. Commands are ready to copy, but read the short explanation beside each one so you know what it does.

- Chapters 1–2 explain the goal and prepare your computer.
- Chapters 3–4 teach every Python tool and every `yt-dlp` setting the program uses, each with a tiny example you can try.
- Chapter 5 builds `downloader.py` step by step until it is complete.
- Chapter 6 runs the program for videos, playlists, and audio, and tests it.
- Chapter 7 shows how real use revealed problems, how their causes were found, and why each fix works.
- Chapter 8 solves common problems and suggests practice ideas.

The English and Persian editions contain the same code, commands, expected results, and advice. Only their language and page direction differ. The complete project and both PDF editions are published on [GitHub](#).

Important

Download only videos that you own, that are offered under a licence that allows it, or that you have permission to save. Respect YouTube's Terms of Service and the creators' copyrights. This tutorial teaches Python; how you use the program is your responsibility.

Contents

1	What We Will Build	1
1.1	The program's behaviour	1
1.2	How it works	1
1.3	Project structure	2
1.4	What you need	2
2	Prepare Python and the Project	3
2.1	Check Python	3
2.2	Install FFmpeg	3
2.3	Get the project folder	4
2.4	Create an isolated environment	4
2.5	Write the requirements file	5
2.6	Install the packages	5
3	The Python Tools We Use	6
3.1	Modules and import	6
3.2	Variables, strings, and in	6
3.3	Functions, return, and default values	7
3.4	Decisions with if and else	7
3.5	Lists, tuples, and dictionaries	7
3.6	Paths with pathlib	8
3.7	Finding a program with shutil.which	8
3.8	The with statement	8
3.9	Command-line arguments with argparse	9
3.10	Asking the user with input	9
3.11	Handling errors with try and except	9
3.12	Replacing text with replace	10
3.13	The if __name__ == "__main__" line	10

4	Meet yt-dlp	11
4.1	A first look from the terminal	11
4.2	Using yt-dlp from Python	11
4.3	The settings our program uses	12
4.3.1	Output templates: naming the files	12
4.3.2	Format selection: choosing the quality	12
4.3.3	Only the sound: post-processors	13
4.3.4	Cookies: when YouTube asks you to sign in	13
5	Write the Program Step by Step	15
5.1	Step 1: describe the file	15
5.2	Step 2: import the tools and set the default folder	15
5.3	Step 3: recognise a playlist	16
5.4	Step 4: choose the file names	16
5.5	Step 5: choose the quality	16
5.6	Step 6: collect all settings	17
5.7	Step 7: download	18
5.8	Step 8: read the command line	18
5.9	Step 9: start the program	19
5.10	The complete program	20
6	Run and Test the Program	23
6.1	Show the help page	23
6.2	Download a single video	23
6.3	Download a whole playlist	24
6.4	Save only the sound	24
6.5	Choose the save folder	25
6.6	Other ways to run it	25
6.7	Stop and continue later	25
6.8	The offline tests	26
7	Lessons from Real Use	28
7.1	A simple method for any bug	28
7.2	Lesson 1: a network drop left half a video	28
7.2.1	The symptom	28
7.2.2	Reading the evidence	29

7.2.3	Finding the cause	29
7.2.4	The fix and the test	29
7.3	Lesson 2: the terminal changed the link	30
7.3.1	The symptom	30
7.3.2	Why it matters	30
7.3.3	The fix	30
7.4	Lesson 3: keep downloads out of the code and out of Git	30
7.5	Lesson 4: a new feature—only the sound	31
7.6	Lesson 5: stopping politely	31
7.7	Summary of the changes	32
8	Troubleshooting and Next Steps	33
8.1	Common problems	34
8.2	Keep the project clean in Git	34
8.3	Ideas for practice	35

What We Will Build

Chapter goal

Understand what the finished program does, how it works in one picture, and which files the project contains.

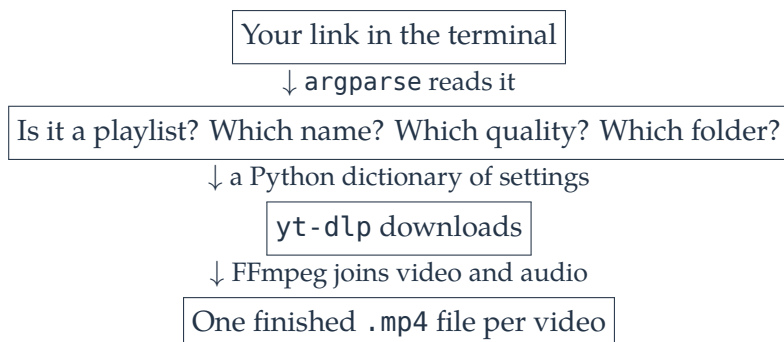
We will write one short Python program, `downloader.py`. You give it a YouTube link in the terminal and it saves the video on your computer.

1.1 The program's behaviour

- **One video link:** the program downloads that video and names the file with its exact YouTube title, for example `Me at the zoo.mp4`.
- **A playlist link:** the program creates a folder with the playlist's title and downloads every video into it. Each file keeps its exact title and gets a number, so the files stay in the playlist order: `01 - First video.mp4`, `02 - Second video.mp4`, ...
- **A save folder:** with `-o FOLDER` you choose where the files go. Without it, files are saved in a `downloads` folder next to `downloader.py`, no matter where you start the program from. Git ignores this folder, so videos never end up in your repository.
- **Only the sound:** with `--audio` the program saves just the audio as an `.mp3` file—useful for music, podcasts, and language lessons.
- **Best quality:** it takes the best video and the best audio and joins them into one `.mp4` file that plays on phones, TVs, and every common player.
- **Friendly and robust:** if you forget the link, it asks for it. If one video in a playlist is private or deleted, it skips that video and continues. After a network drop it retries automatically, and `Ctrl+C` stops it politely so the same command can continue later.

1.2 How it works

The hard work—talking to YouTube, choosing formats, downloading in parts, and showing progress—is done by `yt-dlp`, a popular open-source Python library. Our job is to *prepare the right settings* and hand the link to it.



1.3 Project structure

When you finish, your folder looks like this. Only the files in `code/` are needed to run the program; `docs/` holds this tutorial.

```

prj-youtube-downloader/
|-- code/
|   |-- downloader.py      # the complete program (119 lines)
|   |-- requirements.txt   # the packages to install
|   |-- test_downloader.py # nine offline tests
|   |-- README.md         # how to run the code
|   `-- downloads/        # your videos (created automatically, not in Git)
|-- docs/
|   |-- english/          # this book (English, XeLaTeX)
|   |-- persian/          # the same book in Persian (XePersian)
|   `-- README.md         # how to build the books
|-- .gitignore            # files Git must not save (videos, ...)
|-- LICENSE
`-- README.md             # project overview
  
```

1.4 What you need

- A computer with Windows, macOS, or Linux and an internet connection.
- Python 3.10 or newer (Chapter 2 shows how to check).
- FFmpeg, a free tool for joining video and audio (Chapter 2 installs it).
- A text editor. Visual Studio Code is a good free choice.

You do not need any previous experience with `yt-dlp`. Every Python idea used in the program is explained with an example in Chapter 3.

Checkpoint

You can say in one sentence what happens for a single video link, what happens for a playlist link, and where files go when you do not choose a folder.

Prepare Python and the Project

Chapter goal

Check Python, install FFmpeg, create the project folder and an isolated Python environment, and install the packages.

2.1 Check Python

Open a terminal: *Terminal* on macOS and Linux, *PowerShell* on Windows, or the terminal inside your editor. Then run:

```
python --version
```

On some macOS or Linux systems the command is `python3`. Any result from Python 3.10 upwards, such as Python 3.12.11, is suitable. If Python is missing, install it from <https://www.python.org/downloads/>. On Windows, tick *Add python.exe to PATH* in the installer.

2.2 Install FFmpeg

YouTube stores the best video and the best audio as two separate files. FFmpeg joins them into one .mp4. Install it with your system's package manager:

```
sudo apt install ffmpeg      # Ubuntu / Debian Linux
brew install ffmpeg          # macOS with Homebrew
winget install Gyan.FFmpeg    # Windows PowerShell
```

Close and reopen the terminal, then check it:

```
ffmpeg -version
```

The first line should begin with `ffmpeg version`. The program also works without FFmpeg, but then it can only save lower-quality files that already contain both video and audio (Section 4.3.2 explains why).

2.3 Get the project folder

Choose *one* of the two ways below.

Way A: build everything yourself (recommended for learning)

Create an empty project folder with a code folder inside it, and enter it:

```
mkdir prj-youtube-downloader
cd prj-youtube-downloader
mkdir code
cd code
```

You will type every file in this folder while reading Chapters 2, 5, and 6.

Way B: download the finished project

```
git clone https://github.com/alirezaaies/prj-youtube-downloader.git
cd prj-youtube-downloader/code
```

If you downloaded a ZIP file from GitHub instead, extract it and open a terminal in its code folder.

2.4 Create an isolated environment

A *virtual environment* is a private box of packages for one project. It keeps this project's packages separate from other Python projects, so they never conflict. Use *one* of the following.

Python's built-in venv on macOS or Linux:

```
python3 -m venv .venv
source .venv/bin/activate
```

Python's built-in venv on Windows PowerShell:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Conda (Anaconda or Miniconda), on any system:

```
conda create -n ytdl python=3.12 -y
conda activate ytdl
```

The terminal now shows (`.venv`) or (`ytdl`) before its prompt. Activate the environment again every time you open a new terminal for this project.

Important

If PowerShell refuses to run `Activate.ps1`, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, answer Y, and try again.

2.5 Write the requirements file

A `requirements.txt` file lists the packages a project needs, so anyone can install all of them with one command. In Way A, create `requirements.txt` in the code folder with this content (Way B already has it):

Listing 2.1: `code/requirements.txt`

```
# yt-dlp talks to YouTube; [default] adds its recommended helpers.
# YouTube changes often, so keep yt-dlp up to date (pip install -U ...).
yt-dlp[default]>=2026.8.19
# JavaScript runtime that yt-dlp needs to read YouTube's player.
deno>=2.0
```

- Lines starting with `#` are comments for humans; `pip` ignores them.
- `yt-dlp[default]` installs `yt-dlp` plus its recommended helpers. The part in brackets is called an *extra*.
- `>=2026.8.19` means “this version or any newer one”. We allow newer versions on purpose: YouTube changes often and `yt-dlp` is updated to follow it.
- `deno` is a small JavaScript engine. YouTube’s web player contains JavaScript puzzles, and `yt-dlp` uses `deno` to solve them. Without it, some formats are missing or downloads fail.

2.6 Install the packages

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The first line updates `pip`, Python’s package installer. The second reads `requirements.txt` (`-r`) and installs everything listed in it. Writing `python -m pip` instead of only `pip` guarantees that the `pip` of the *active* environment is used.

Notice a small naming difference: the package is installed as `yt-dlp` (with a hyphen) but imported in Python as `yt_dlp` (with an underscore), because Python names cannot contain hyphens.

Checkpoint

Run the three commands below. Each prints a version number. If all three work, your environment is ready.

```
python -c "import yt_dlp; print(yt_dlp.version.__version__)"
deno --version
ffmpeg -version
```

The Python Tools We Use

Chapter goal

Learn every Python idea that appears in `downloader.py`, each with a tiny example you can run yourself.

You can try every example in the *Python interactive shell*. Type `python` in the activated terminal; the prompt changes to `>>>`. Type a line and press Enter to see its result. Leave the shell with `exit()`. Lines that begin with `#` in the examples are comments, and the expected output is shown in a comment such as `# -> True`.

3.1 Modules and import

A *module* is a file of ready-made Python code. `import` makes it available in your program. Some modules come with Python (the *standard library*); others, such as `yt_dlp`, are installed with `pip`.

```
import shutil          # use it as shutil.which(...)
from pathlib import Path # take only Path from the pathlib module
from yt_dlp import YoutubeDL # take only YoutubeDL from yt_dlp
```

Our program uses three standard modules—`argparse` (read the command line), `shutil` (find programs), and `pathlib` (work with folders)—and one installed library, `yt_dlp`.

3.2 Variables, strings, and in

A *variable* is a name for a value. Text values are *strings* and are written in quotes. The `in` operator checks whether one string appears inside another and gives `True` or `False` (a *boolean*).

```
url = "https://www.youtube.com/playlist?list=PL123"
print("list=" in url)          # -> True
print("list=" in "watch?v=abc") # -> False
```

An *f-string* (a string with `f` before the quote) puts values inside text with curly braces:

```
folder = "/home/sara/Videos"
print(f"Saving to: {folder}") # -> Saving to: /home/sara/Videos
```

The method `.strip()` removes spaces and line breaks from both ends of a string. This is useful for text that a user pastes:

```
print(" https://youtu.be/abc ".strip()) # -> https://youtu.be/abc
```

3.3 Functions, return, and default values

A *function* is a named block of code that you can call again and again. `def` creates it; `return` sends a result back to the caller. The string right below the `def` line is a *docstring*: a short description for humans. Indented lines (four spaces) belong to the function.

```
def greet(name, greeting="Hello"):
    """Build a short greeting."""
    return f"{greeting}, {name}!"

print(greet("Sara"))          # -> Hello, Sara!
print(greet("Sara", "Hi"))    # -> Hi, Sara!
```

`greeting="Hello"` is a *default value*: it is used when the caller does not give that argument. Our download function uses the same idea for its folder and browser. The special value `None` means “nothing was given”.

3.4 Decisions with `if` and `else`

`if` runs its block only when a condition is true; `else` runs otherwise. A `return` ends the function immediately, so code after it does not run.

```
def size_word(count):
    if count > 1:
        return "many videos"
    return "one video"

print(size_word(5))    # -> many videos
print(size_word(1))    # -> one video
```

Python also treats empty values as false: `""`, `None`, and `0` are false; a non-empty string is true. So `if browser:` means “if a browser name was given”. The `or` operator returns its first true value:

```
print(None or "typed by user")    # -> typed by user
print("from command" or "typed")  # -> from command
```

3.5 Lists, tuples, and dictionaries

- A *list* holds values in order and can change: `["a", "b"]`.
- A *tuple* is like a list but cannot change: `("firefox",)`. A tuple with one value needs the trailing comma; `("firefox")` is just a string.
- A *dictionary* stores *key: value* pairs, like a small form with labelled fields.

```
options = {"format": "best", "ignoreerrors": True}
print(options["format"])          # -> best
options["cookiesfrombrowser"] = ("firefox",) # add a new key
print(len(options))                # -> 3
```

yt-dlp receives all of its settings as one dictionary, so this is the most important data type in our program.

3.6 Paths with pathlib

Path represents a file or folder path and works the same on Windows, macOS, and Linux.

```
from pathlib import Path

p = Path("~/Videos").expanduser() # ~ becomes your home folder
print(p)                          # -> /home/sara/Videos
print(p / "Music")                 # -> /home/sara/Videos/Music
print(Path("code/downloader.py").resolve().parent) # full folder path
p.mkdir(parents=True, exist_ok=True) # create it (and its parents) if missing
```

- `__file__` is a special variable that holds the path of the running Python file.
- `.resolve()` turns a path into a complete (absolute) path.
- `.parent` is the folder that contains the path.
- `exist_ok=True` means “no error if the folder already exists”.

So `Path(__file__).resolve().parent` is “the folder that contains this script”. That is exactly our default save folder.

3.7 Finding a program with shutil.which

`shutil.which(name)` looks for a command on your computer. It returns the command’s path when it exists, and `None` when it does not:

```
import shutil
print(shutil.which("ffmpeg"))      # -> /usr/bin/ffmpeg
print(shutil.which("no-such-tool")) # -> None
```

3.8 The with statement

Some objects must be opened and later closed properly. `with` does both: it opens the object, runs the indented block, and closes it at the end—even if an error happens.

```
with open("note.txt", "w") as f: # open a file for writing
    f.write("Hello")
# here the file is already closed
```

We use `with YoutubeDL(...)` as `yd1`: in the same way.

3.9 Command-line arguments with argparse

argparse reads the words typed after `python script.py`. It also creates a `--help` page and shows clear errors automatically. Save this as `hello.py`:

```
import argparse

parser = argparse.ArgumentParser(description="Say hello.")
parser.add_argument("name", nargs="?", help="who to greet")
parser.add_argument("-t", "--times", default="1", help="how many times")
args = parser.parse_args()
print(f"Hello {args.name}! " * int(args.times))
```

```
python hello.py Sara          # -> Hello Sara!
python hello.py Sara -t 2     # -> Hello Sara! Hello Sara!
python hello.py --help       # shows the automatic help page
```

- "name" has no dashes, so it is a *positional* argument: its value is given by position.
- `nargs="?"` makes it optional; when it is missing its value is `None`.
- `-t`, `--times` is an *option*: a short and a long name for the same setting, with a default value.
- After `parse_args()`, values are available as `args.name` and `args.times`.

Some options are simple on/off switches that take no value. They are created with the setting `action="store_true"`. The value is then `True` when the option is written and `False` when it is not.

```
parser.add_argument("-l", "--loud", action="store_true", help="shout")
# python hello.py Sara          -> args.loud is False
# python hello.py Sara --loud   -> args.loud is True
```

3.10 Asking the user with input

`input(message)` shows a message, waits until the user presses Enter, and returns the typed text as a string:

```
link = input("Paste a link: ").strip()
print(f"You pasted {link}")
```

3.11 Handling errors with try and except

When something goes wrong while a program runs, Python raises an *exception* and, if nobody handles it, stops with a long error report (a *traceback*). `try` runs a block; if a named exception happens inside it, the matching `except` block runs instead of the crash.

```
try:
    number = int(input("Type a number: "))
except ValueError:
    print("That was not a number.")
```

Pressing Ctrl+C in the terminal raises the exception `KeyboardInterrupt`. Our program catches it to print one friendly sentence instead of a traceback. Catch only the exceptions you expect; catching every error would also hide real bugs.

3.12 Replacing text with `replace`

`text.replace(old, new)` returns a copy of the string in which every `old` becomes `new`. Replacing with an empty string removes it. In Python code a single backslash is written `"\"`, because one backslash alone starts an *escape sequence* such as `\n` (new line).

```
link = "https://youtu.be/abc\\?si\\=xyz"
print(link)                # -> https://youtu.be/abc\?si\=xyz
print(link.replace("\\", "")) # -> https://youtu.be/abc?si=xyz
```

3.13 The `if __name__ == "__main__"` line

When you run a file directly, Python sets its `__name__` variable to `"__main__"`. When another file imports it (for example our test file), `__name__` is the module's name instead. So this block means "start the program only when this file is run directly":

```
if __name__ == "__main__":
    main()
```

Thanks to it, the tests can import our functions without starting a download.

Try it yourself

Open the Python shell and run the `pathlib` and `shutil`. which examples on your own computer. Compare your results with the comments.

Checkpoint

You can explain what a function, a dictionary, `Path(__file__).resolve().parent`, `shutil.which`, `argparse`, and `try/except` are, in one sentence each.

Meet yt-dlp

Chapter goal

Understand how yt-dlp is used from Python and what each setting in our options dictionary does.

yt-dlp is a free, open-source downloader for YouTube and many other sites (<https://github.com/yt-dlp/yt-dlp>). It can be used in two ways: as a terminal command and as a Python library. Our program uses the library, but trying the command first helps you see what it does.

4.1 A first look from the terminal

```
yt-dlp --version
yt-dlp -F "https://www.youtube.com/watch?v=jNQXAC9IVRw"
```

The second command does not download anything. -F lists every *format* YouTube offers for that video. You will see audio-only rows, video-only rows, and a few rows with both. Notice that the best qualities are always *video only* or *audio only*. That is why we need FFmpeg to join them.

Important

Always put a YouTube link inside quotes in the terminal. Links often contain &, which the terminal would otherwise treat as a command separator.

4.2 Using yt-dlp from Python

From Python, yt-dlp needs only three steps: write a settings dictionary, create a YoutubeDL object with it, and call download with a *list* of links.

```
from yt_dlp import YoutubeDL

options = {"outtmpl": "%(title)s.%(ext)s"}
with YoutubeDL(options) as ydl:
    result = ydl.download(["https://www.youtube.com/watch?v=jNQXAC9IVRw"])
print(result)  # -> 0 means success
```

download returns 0 when every link succeeded and another number when something failed. Our program uses this number to print its final message.

4.3 The settings our program uses

Key	What it does
format	Which video and audio to download.
format_sort	Which formats to prefer when several are “best”.
merge_output_format	The container used when video and audio are joined: mp4.
paths	The base folder for downloads: {"home": folder}.
outtmpl	The <i>output template</i> : how each file is named.
ignoreerrors	True: skip a failing video and continue with the rest.
retries	Try again up to 10 times after a network drop.
postprocessors	Work done <i>after</i> a download, such as converting to MP3.
cookiesfrombrowser	Use the YouTube login of a browser: ("firefox",).

4.3.1 Output templates: naming the files

An output template is a normal string with special fields of the form `%(field)s`. `yt-dlp` replaces each field with information about the video:

Field	Replaced with
<code>%(title)s</code>	the exact video title
<code>%(ext)s</code>	the file extension, such as mp4
<code>%(playlist_title)s</code>	the playlist title
<code>%(playlist_index)02d</code>	the position in the playlist, as two digits

`s` means “as text” and `02d` means “as a whole number with at least two digits, padded with zero”, so 3 becomes 03. Padding keeps files sorted correctly in your file manager (02 before 10).

A `/` in the template creates a folder. So for a playlist named *Python Basics* whose third video is *Loops*, the template

```
%(playlist_title)s/%(playlist_index)02d - %(title)s.%(ext)s
```

produces `Python Basics/03 - Loops.mp4`. If a title contains characters that are not allowed in file names (such as `/` or `:` on Windows), `yt-dlp` replaces them safely by itself. Persian and other non-English titles are kept as they are.

4.3.2 Format selection: choosing the quality

The format value is a small expression:

```
bestvideo*+bestaudio/best
```

- `bestvideo*` is the best video stream; the `*` also allows streams that happen to contain audio.
- `+bestaudio` adds the best audio stream; the two are joined by FFmpeg.
- `/best` is a fallback: “if that is impossible, take the best single file that already contains video and audio”.

Joining needs FFmpeg. Without FFmpeg we ask only for `best`, which never needs joining. The best single files on YouTube are usually only 360p, so FFmpeg is strongly recommended.

`format_sort` decides which of several good formats wins. YouTube often offers modern codecs (AV1, VP9, Opus) that some TVs, phones, and older players cannot open. The list

```
[ "vcodec:h264", "res", "acodec:m4a" ]
```

means: prefer H.264 video, then the highest resolution, then AAC (m4a) audio. The result is an .mp4 file that plays everywhere, normally in full HD (1080p).

4.3.3 Only the sound: post-processors

For audio only, the format is simply `bestaudio/best`: the best audio stream, or the best file if no separate audio exists. YouTube’s best audio is an .m4a (AAC) or .webm (Opus) file. To get an .mp3, which every phone, car, and old player understands, we ask for a *post-processor*: a step that `yt-dlp` runs after the download.

```
"postprocessors": [
    { "key": "FFmpegExtractAudio", "preferredcodec": "mp3",
      "preferredquality": "192"},
]
```

- `postprocessors` is a *list* because several steps can run one after another; each step is a dictionary.
- `"key"` names the step. `FFmpegExtractAudio` asks FFmpeg to convert the downloaded file to audio.
- `preferredcodec` chooses the output format and `preferredquality` the bit rate in kbit/s: 192 is clear speech and good music at a moderate size (about 1.4 MB per minute).

After the conversion `yt-dlp` deletes the original .m4a, so only the .mp3 is left. The output template stays the same, so an audio file gets the same exact title, and a playlist gets the same numbered folder.

4.3.4 Cookies: when YouTube asks you to sign in

Sometimes YouTube answers with *Sign in to confirm you’re not a bot*. This happens more often with VPNs and shared networks. A *cookie* is a small piece of data that a website stores in your browser to remember your login. With `cookiesfrombrowser`, `yt-dlp` reads the YouTube cookies of a browser on your own computer and uses your login, exactly as the browser would.

Important

Cookies are as private as a password. The program only reads them on your own computer; it never saves or sends them anywhere else. Never share cookie files and never put them in Git.

Checkpoint

Without running anything, write down the file name that the template `%(playlist_index)02d - %(title)s.%(ext)s` produces for the 7th video, titled *Functions*, saved as mp4. Answer: `07 - Functions.mp4`.

Write the Program Step by Step

Chapter goal

Create `downloader.py` piece by piece and understand every line. At the end of this chapter the program is complete.

Create an empty file named `downloader.py` in the code folder and type each piece below into it, in order. The line numbers in the margin are the line numbers of the finished file, so you can always see where a piece belongs. Leave two empty lines between functions; this is the usual Python style (PEP 8).

5.1 Step 1: describe the file

```

1  """Download one YouTube video or a whole playlist with yt-dlp.
2
3  Examples:
4      python downloader.py "VIDEO_OR_PLAYLIST_LINK"
5      python downloader.py "VIDEO_OR_PLAYLIST_LINK" -o ~/Videos
6      python downloader.py "VIDEO_OR_PLAYLIST_LINK" --audio
7      python downloader.py "VIDEO_OR_PLAYLIST_LINK" --browser firefox
8      python downloader.py          # asks for the link
9  """

```

A string at the very top of a file is the *module docstring*. Python ignores it while running, but it tells every reader what the file does and how to use it.

5.2 Step 2: import the tools and set the default folder

```

11 import argparse
12 import shutil
13 from pathlib import Path
14
15 from yt_dlp import YoutubeDL
16
17 # When no folder is given, files go to a "downloads" folder next to this script.
18 SCRIPT_FOLDER = Path(__file__).resolve().parent
19 DOWNLOAD_FOLDER = SCRIPT_FOLDER / "downloads"

```

- `argparse` will read the command line, `shutil` will look for `FFmpeg`, and `Path` will handle folders (Chapter 3).
- `YoutubeDL` is the downloader class from `yt_dlp` (Chapter 4).
- `SCRIPT_FOLDER` is the folder that contains `downloader.py`.
- `DOWNLOAD_FOLDER` is a `downloads` folder inside it. The `/` operator joins two parts of a path (Chapter 3). The folder is created automatically by the first download.

Names in capital letters are a Python convention for values that never change (*constants*). Using the script's folder instead of the terminal's current folder means the result is always in the same predictable place, even if you start the program from somewhere else, for example with `python code/downloader.py`. Chapter 7 explains why the files go into their own downloads folder.

5.3 Step 3: recognise a playlist

```

22 def is_playlist(url):
23     """Return True when the link contains a playlist id (list=...)."""
24     return "list=" in url

```

Every YouTube playlist link contains `list=` followed by the playlist's id:

<code>https://www.youtube.com/playlist?list=PLxxxxxxx</code>	-> playlist
<code>https://www.youtube.com/watch?v=abc123&list=PLxxxxxxx</code>	-> playlist
<code>https://www.youtube.com/watch?v=abc123</code>	-> one video
<code>https://youtu.be/abc123</code>	-> one video

The expression `"list=" in url` is already `True` or `False`, so we can return it directly. A link to a video that was opened *from* a playlist also contains `list=`; in that case the whole playlist is downloaded. To download only that single video, copy the link without the `&list=...` part.

5.4 Step 4: choose the file names

```

27 def file_name_template(url):
28     """Tell yt-dlp how to name each downloaded file."""
29     if is_playlist(url):
30         # Playlist title/01 - Video title.mp4
31         return "%(playlist_title)s/%(playlist_index)02d - %(title)s.%(ext)s"
32     # Video title.mp4
33     return "%(title)s.%(ext)s"

```

This function returns one of the two output templates from Chapter 4. For a playlist, the `/` creates a folder with the playlist title and the number keeps the original order. For a single video, the file is simply named after its title. Notice the early return: if the link is a playlist, the function ends on line 31, so no `else` is needed.

5.5 Step 5: choose the quality

```

36 def video_quality():
37     """Choose the best quality that this computer can save."""
38     if shutil.which("ffmpeg"):
39         # Best video + best audio, joined into one file by ffmpeg.
40         return "bestvideo*+bestaudio/best"
41     # Without ffmpeg, take the best file that already contains both.
42     return "best"

```

`shutil.which("ffmpeg")` returns a path when FFmpeg is installed and `None` when it is not. With FFmpeg we ask for the best video plus the best audio; without it we ask for the best single file, so the program still works instead of stopping with an error.

5.6 Step 6: collect all settings

```

45 def build_options(url, folder, browser=None, audio=False):
46     """Collect all settings in the dictionary that yt-dlp expects."""
47     options = {
48         "format": video_quality(),
49         # Prefer H.264 video and AAC (m4a) audio: an mp4 that plays everywhere.
50         "format_sort": ["vcodec:h264", "res", "acodec:m4a"],
51         "merge_output_format": "mp4",
52         "paths": {"home": str(folder)},
53         "outtmpl": file_name_template(url),
54         "ignoreerrors": True, # skip a private or deleted playlist video
55         # Retry after a network drop (the Python default is 0 retries).
56         "retries": 10,
57     }
58     if audio:
59         # Only the sound: the best audio stream, converted to mp3 by ffmpeg.
60         options["format"] = "bestaudio/best"
61         if shutil.which("ffmpeg"):
62             options["postprocessors"] = [
63                 {"key": "FFmpegExtractAudio", "preferredcodec": "mp3",
64                  "preferredquality": "192"},
65             ]
66     if browser:
67         # Reuse the YouTube login of this browser when YouTube asks for it.
68         options["cookiesfrombrowser"] = (browser,)
69     return options

```

This function builds the dictionary that YoutubeDL expects. Each key was explained in the tables in Chapter 4.

- The values of `format` and `outtmpl` come from the two small functions we just wrote. Small functions with clear names make the main settings easy to read.
- `paths` receives the folder as a string (`str(folder)`), because that is what `yt-dlp` expects.
- `retries` is important. The `yt-dlp` *command* retries 10 times by default, but from *Python* the default is 0. Without this line, one short network drop stops the video download (Chapter 7 tells the full story).
- Lines 58–64 run only when `audio` is `True`. They *replace* the `format` with `bestaudio/best` (only the sound) and, when FFmpeg exists, add a *post-processor* that converts the result to an

MP3 file at 192 kbit/s. Without FFmpeg the original audio file (usually .m4a) is kept, which most players can also open.

- `browser=None` and `audio=False` are default values: when the caller does not give them, no cookie setting is added (line 66) and a full video is downloaded.
- `(browser,)` is a tuple with one item; `yt-dlp` expects a tuple here because it can also accept a profile name as a second item.

Building the dictionary first and then changing a few keys with `if` blocks is a common pattern: the normal case stays readable, and each special case is one small, separate block.

5.7 Step 7: download

```

72 def download(url, folder=DOWNLOAD_FOLDER, browser=None, audio=False):
73     """Download the link into folder. Return 0 when everything succeeded."""
74     folder = Path(folder).expanduser().resolve()
75     folder.mkdir(parents=True, exist_ok=True)
76     print(f"Saving to: {folder}")
77
78     with YoutubeDL(build_options(url, folder, browser, audio)) as ydl:
79         return ydl.download([url])

```

1. Line 74 turns whatever the user typed into a complete path: `expanduser` replaces `~` with the home folder, and `resolve` makes a relative path such as `videos` absolute.
2. Line 75 creates the folder if it does not exist yet.
3. Line 76 prints the exact place, so the user always knows where to look.
4. Lines 78–79 create the downloader with our settings, download the link, and return `yt-dlp`'s result: `0` for complete success.

5.8 Step 8: read the command line

```

82 def main():
83     parser = argparse.ArgumentParser(
84         description="Download a YouTube video or a whole playlist."
85     )
86     parser.add_argument("url", nargs="?", help="video or playlist link")
87     parser.add_argument(
88         "-o", "--output", default=DOWNLOAD_FOLDER,
89         help="folder for the files (default: downloads/ next to this script)",
90     )
91     parser.add_argument(
92         "-a", "--audio", action="store_true",
93         help="save only the sound, as an mp3 file",
94     )
95     parser.add_argument(
96         "--browser",
97         help="use this browser's YouTube login, e.g. firefox or chrome",

```

```

98     )
99     args = parser.parse_args()
100
101     url = args.url or input("Paste a YouTube video or playlist link: ").strip()
102     url = url.replace("\\", "") # zsh may paste ? and = as \? and \=
103     if not url:
104         parser.error("a link is required")
105
106     try:
107         result = download(url, args.output, args.browser, args.audio)
108     except KeyboardInterrupt:
109         print("\nStopped. Run the same command again to continue.")
110         return
111
112     if result == 0:
113         print("Done. All files were downloaded.")
114     else:
115         print("Finished, but some files could not be downloaded (see the errors above).")

```

main connects the user to the functions above.

- url is positional and optional (nargs="?").
- -o / --output chooses the folder; its default is DOWNLOAD_FOLDER.
- -a / --audio uses action="store_true": it takes no value. Writing --audio makes args.audio True; leaving it out makes it False (Chapter 3).
- --browser is optional; when it is not given, its value is None.
- Line 101: if no link was typed after the command, args.url is None, so or calls input and asks for one.
- Line 102 removes every backslash. When you paste a link into zsh (the default terminal shell on macOS and many Linux systems), it may add \ before ? and =. A real link never contains a backslash, so removing them is always safe (Chapter 7).
- Lines 103–104: if the user only pressed Enter, parser.error prints a clear message and stops the program.
- Lines 106–110 run the download inside try. If the user presses Ctrl+C, Python raises KeyboardInterrupt; instead of a long error report, the program prints one friendly sentence and return ends main.
- Lines 112–115 print a final message based on the result.

5.9 Step 9: start the program

```

118 if __name__ == "__main__":
119     main()

```

These two lines call main only when the file is run directly, not when the test file imports it (Chapter 3).

5.10 The complete program

Your file should now be identical to the listing below. Compare them line by line if anything does not work.

Listing 5.1: The complete code in code/downloader.py

```

1  """Download one YouTube video or a whole playlist with yt-dlp.
2
3  Examples:
4      python downloader.py "VIDEO_OR_PLAYLIST_LINK"
5      python downloader.py "VIDEO_OR_PLAYLIST_LINK" -o ~/Videos
6      python downloader.py "VIDEO_OR_PLAYLIST_LINK" --audio
7      python downloader.py "VIDEO_OR_PLAYLIST_LINK" --browser firefox
8      python downloader.py          # asks for the link
9  """
10
11 import argparse
12 import shutil
13 from pathlib import Path
14
15 from yt_dlp import YoutubeDL
16
17 # When no folder is given, files go to a "downloads" folder next to this script.
18 SCRIPT_FOLDER = Path(__file__).resolve().parent
19 DOWNLOAD_FOLDER = SCRIPT_FOLDER / "downloads"
20
21
22 def is_playlist(url):
23     """Return True when the link contains a playlist id (list=...)."""
24     return "list=" in url
25
26
27 def file_name_template(url):
28     """Tell yt-dlp how to name each downloaded file."""
29     if is_playlist(url):
30         # Playlist title/01 - Video title.mp4
31         return "%(playlist_title)s/%(playlist_index)02d - %(title)s.%(ext)s"
32     # Video title.mp4
33     return "%(title)s.%(ext)s"
34
35
36 def video_quality():
37     """Choose the best quality that this computer can save."""
38     if shutil.which("ffmpeg"):
39         # Best video + best audio, joined into one file by ffmpeg.
40         return "bestvideo*+bestaudio/best"
41     # Without ffmpeg, take the best file that already contains both.
42     return "best"
43
44
45 def build_options(url, folder, browser=None, audio=False):
46     """Collect all settings in the dictionary that yt-dlp expects."""
47     options = {
48         "format": video_quality(),
49         # Prefer H.264 video and AAC (m4a) audio: an mp4 that plays everywhere.
50         "format_sort": ["vcodec:h264", "res", "acodec:m4a"],

```

```

51     "merge_output_format": "mp4",
52     "paths": {"home": str(folder)},
53     "outtmpl": file_name_template(url),
54     "ignoreerrors": True, # skip a private or deleted playlist video
55     # Retry after a network drop (the Python default is 0 retries).
56     "retries": 10,
57 }
58 if audio:
59     # Only the sound: the best audio stream, converted to mp3 by ffmpeg.
60     options["format"] = "bestaudio/best"
61     if shutil.which("ffmpeg"):
62         options["postprocessors"] = [
63             {"key": "FFmpegExtractAudio", "preferredcodec": "mp3",
64              "preferredquality": "192"},
65         ]
66 if browser:
67     # Reuse the YouTube login of this browser when YouTube asks for it.
68     options["cookiesfrombrowser"] = (browser,)
69 return options
70
71
72 def download(url, folder=DOWNLOAD_FOLDER, browser=None, audio=False):
73     """Download the link into folder. Return 0 when everything succeeded."""
74     folder = Path(folder).expanduser().resolve()
75     folder.mkdir(parents=True, exist_ok=True)
76     print(f"Saving to: {folder}")
77
78     with YoutubeDL(build_options(url, folder, browser, audio)) as ydl:
79         return ydl.download([url])
80
81
82 def main():
83     parser = argparse.ArgumentParser(
84         description="Download a YouTube video or a whole playlist."
85     )
86     parser.add_argument("url", nargs="?", help="video or playlist link")
87     parser.add_argument(
88         "-o", "--output", default=DOWNLOAD_FOLDER,
89         help="folder for the files (default: downloads/ next to this script)",
90     )
91     parser.add_argument(
92         "-a", "--audio", action="store_true",
93         help="save only the sound, as an mp3 file",
94     )
95     parser.add_argument(
96         "--browser",
97         help="use this browser's YouTube login, e.g. firefox or chrome",
98     )
99     args = parser.parse_args()
100
101     url = args.url or input("Paste a YouTube video or playlist link: ").strip()
102     url = url.replace("\\", "") # zsh may paste ? and = as \? and \=
103     if not url:
104         parser.error("a link is required")
105
106     try:
107         result = download(url, args.output, args.browser, args.audio)

```

```
108     except KeyboardInterrupt:
109         print("\nStopped. Run the same command again to continue.")
110         return
111
112     if result == 0:
113         print("Done. All files were downloaded.")
114     else:
115         print("Finished, but some files could not be downloaded (see the errors
116         above).")
117
118 if __name__ == "__main__":
119     main()
```

Checkpoint

Run `python downloader.py --help` in the code folder. You should see the usage line and four arguments: `url`, `-o OUTPUT`, `-a`, and `--browser BROWSER`. If Python shows a `SyntaxError` or an `IndentationError`, compare the line it names with the listing above.

Run and Test the Program

Chapter goal

Download a single video and a playlist, choose a save folder, and run the offline tests.

Make sure the environment is active (Chapter 2) and the terminal is in the code folder. On some systems you must type `python3` instead of `python`.

6.1 Show the help page

```
python downloader.py --help
```

Expected result

```
usage: downloader.py [-h] [-o OUTPUT] [-a] [--browser BROWSER] [url]

Download a YouTube video or a whole playlist.

positional arguments:
  url                  video or playlist link

options:
  -h, --help            show this help message and exit
  -o OUTPUT, --output OUTPUT
                        folder for the files (default: downloads/ next to
                        this
                        script)
  -a, --audio            save only the sound, as an mp3 file
  --browser BROWSER     use this browser's YouTube login, e.g. firefox or
                        chrome
```

6.2 Download a single video

We use the first video ever uploaded to YouTube because it is only 19 seconds long:

```
python downloader.py "https://www.youtube.com/watch?v=jNQXAC9IVRw"
```

yt-dlp prints what it is doing and a live progress line. The important lines look like this (paths are shortened):

Expected result

```
Saving to: /home/sara/prj-youtube-downloader/code/downloads
[youtube] Extracting URL: https://www.youtube.com/watch?v=jNQXAC9IVRw
[info] jNQXAC9IVRw: Downloading 1 format(s): 18
[download] Destination: .../code/downloads/Me at the zoo.mp4
[download] 100% of ...
Done. All files were downloaded.
```

The file `Me at the zoo.mp4` is now in `code/downloads/`. This old video has no HD version, so one ready-made file (format 18) was enough. For an HD video you will see two format numbers, such as 137+140, and an extra line `[Merger] Merging formats into ...`: that is FFmpeg joining the separate video and audio. Format numbers may differ on your computer; that is normal.

6.3 Download a whole playlist

Open any YouTube playlist, copy the link from the address bar, and run:

```
python downloader.py "https://www.youtube.com/playlist?list=PLAYLIST_ID"
```

Expected result

A new folder with the playlist's title appears in `code/downloads/`. The videos inside it are numbered in playlist order and keep their exact titles, for example:

```
Get more creative with AI/
|-- 01 - Vibe coding in Google AI Studio is changing the game.mp4
|-- 02 - 3 Incredible #NanoBanana Community Builds.mp4
`-- ...
```

A large playlist can take a long time. You can stop at any moment with `Ctrl+C`. If you run the same command again later, `yt-dlp` skips the videos that are already complete and continues with the rest.

6.4 Save only the sound

Add `-a` or `--audio`. It works for single videos and for playlists:

```
python downloader.py "https://youtu.be/huwi-cjPPXU" --audio
```

Expected result

```
[info] huwi-cjPPXU: Downloading 1 format(s): 140
[download] 100% of 14.00MiB in 00:00:04 at 2.98MiB/s
[ExtractAudio] Destination: ../code/downloads/Introduce Yourself
                    in Slow German (for Absolute Beginners).mp3
Deleting original file ... .m4a (pass -k to keep)
Done. All files were downloaded.
```

Format 140 is YouTube's AAC audio. The ExtractAudio line is FFmpeg converting it to MP3; then the temporary .m4a is deleted. This 15-minute lesson becomes a 21 MB MP3 file. Only 14 MB had to be downloaded, instead of 144 MB for the full HD video.

6.5 Choose the save folder

Add -o and a folder. The folder is created if it does not exist:

```
python downloader.py "LINK" -o ~/Videos           # macOS / Linux
python downloader.py "LINK" -o "D:\My Videos"    # Windows
python downloader.py "LINK" -o downloads          # a folder here
```

The first line of the output, Saving to: ..., always shows the complete path. Put the folder in quotes when its name contains spaces.

6.6 Other ways to run it

```
python downloader.py                               # asks for the link
python downloader.py "LINK" --browser firefox      # use Firefox's YouTube login
python downloader.py "LINK" --browser chrome -o ~/Videos
```

Without a link, the program shows Paste a YouTube video or playlist link:. Paste the link (usually Ctrl+Shift+V in a Linux terminal, right-click in Windows) and press Enter. Use --browser only when YouTube asks you to sign in (Chapter 8). Supported names include firefox, chrome, chromium, edge, brave, opera, and safari.

6.7 Stop and continue later

Press Ctrl+C at any moment. The program answers:

Expected result

```
Stopped. Run the same command again to continue.
```

The unfinished part stays in the folder as a .part file. When you run the same command again, yt-dlp prints Resuming download at byte ... and continues from there instead of starting over.

6.8 The offline tests

Tests are small programs that check other code automatically. Ours run in a fraction of a second and need no internet, because they check only how the program *prepares* a download. Create `test_downloader.py` next to `downloader.py`:

Listing 6.1: The complete code in `code/test_downloader.py`

```

1  """Offline checks for downloader.py. Run with: python -m unittest -v"""
2
3  import unittest
4  from pathlib import Path
5  from unittest.mock import patch
6
7  import downloader
8
9  VIDEO = "https://www.youtube.com/watch?v=abc123"
10 PLAYLIST = "https://www.youtube.com/playlist?list=PL123"
11
12
13 class DownloaderTests(unittest.TestCase):
14     def test_detects_playlist_and_single_video(self):
15         self.assertTrue(downloader.is_playlist(PLAYLIST))
16         self.assertFalse(downloader.is_playlist(VIDEO))
17
18     def test_single_video_is_named_by_title(self):
19         self.assertEqual(downloader.file_name_template(VIDEO), "%(title)s.%(ext)s"
20 )
21
22     def test_playlist_gets_its_own_folder(self):
23         name = downloader.file_name_template(PLAYLIST)
24         self.assertTrue(name.startswith("%(playlist_title)s/"))
25
26     def test_default_folder_is_downloads_next_to_script(self):
27         script_folder = Path(downloader.__file__).resolve().parent
28         self.assertEqual(downloader.DOWNLOAD_FOLDER, script_folder / "downloads")
29
30     def test_options_use_given_folder_and_browser(self):
31         options = downloader.build_options(VIDEO, "/tmp/videos", "firefox")
32         self.assertEqual(options["paths"], {"home": "/tmp/videos"})
33         self.assertEqual(options["cookiesfrombrowser"], ("firefox",))
34
35     def test_retries_after_network_drop(self):
36         self.assertEqual(downloader.build_options(VIDEO, "/tmp/videos")["retries"
37 ], 10)
38
39     def test_audio_only_becomes_mp3(self):
40         with patch("shutil.which", return_value="/usr/bin/ffmpeg"):
41             options = downloader.build_options(VIDEO, "/tmp/videos", audio=True)
42             self.assertEqual(options["format"], "bestaudio/best")
43             self.assertEqual(options["postprocessors"][0]["preferredcodec"], "mp3")
44
45     def test_audio_only_without_ffmpeg_keeps_original_file(self):
46         with patch("shutil.which", return_value=None):
47             options = downloader.build_options(VIDEO, "/tmp/videos", audio=True)
48             self.assertEqual(options["format"], "bestaudio/best")
49             self.assertNotIn("postprocessors", options)

```

```

48
49     def test_quality_without_ffmpeg(self):
50         with patch("shutil.which", return_value=None):
51             self.assertEqual(downloader.video_quality(), "best")
52
53
54 if __name__ == "__main__":
55     unittest.main()

```

- unittest is Python's built-in testing module. Each method whose name starts with `test_` inside a `unittest.TestCase` class is one test.
- `assertTrue`, `assertFalse`, and `assertEqual` check a result; if a check is wrong, the test fails and shows both values.
- `patch` temporarily replaces a function. In the last test, `shutil.which` pretends that FFmpeg is missing, so we can test that case even on a computer that has FFmpeg. The two audio tests use the same trick in both directions: FFmpeg present (MP3 conversion is added) and FFmpeg missing (the original audio file is kept).

Run all tests:

```
python -m unittest -v
```

Expected result

```

test_audio_only_becomes_mp3 ... ok
test_audio_only_without_ffmpeg_keeps_original_file ... ok
test_default_folder_is_downloads_next_to_script ... ok
test_detects_playlist_and_single_video ... ok
test_options_use_given_folder_and_browser ... ok
test_playlist_gets_its_own_folder ... ok
test_quality_without_ffmpeg ... ok
test_retries_after_network_drop ... ok
test_single_video_is_named_by_title ... ok
-----
Ran 9 tests in 0.001s

OK

```

Whenever you change `downloader.py`, run the tests again. If you change a behaviour on purpose, update the matching test too.

Checkpoint

You downloaded one video to `code/downloads`, saved one audio file with `--audio`, one playlist (or part of it) into its own folder, used `-o` once, and saw `OK` from the tests.

Lessons from Real Use

Chapter goal

Follow how the first version of the program was improved after it was used on real videos: what went wrong, how the cause was found, why each fix works, and how it was tested. These are the habits that turn a working script into a reliable tool.

The first version of `downloader.py` passed all its tests and worked on short videos. Real use then showed problems that no test had covered. This chapter tells that story, because *finding* a problem teaches more than any finished code. Every change described here is already part of the program in Chapter 5.

7.1 A simple method for any bug

1. **Describe the symptom exactly.** What did you expect, and what happened?
2. **Read the whole output**, especially the first warning or error.
3. **Look at the evidence:** which files exist, how large they are, what their names say.
4. **Form a guess about the cause, then check it** in the documentation or the library's source code. Do not change code on a guess alone.
5. **Fix the cause, not the symptom**, with the smallest clear change.
6. **Add a test** that would have caught the problem, and repeat the real case.

7.2 Lesson 1: a network drop left half a video

7.2.1 The symptom

A learner downloaded a 15-minute, full HD lesson with this command:

```
python downloader.py "https://youtu.be/huwi-cjPPXU?si\=Huhcwi9PkWR87c-u" -o ~/Videos
```

The output contained one error, and the program ended with the “some files could not be downloaded” message:

```
[info] huwi-cjPPXU: Downloading 1 format(s): 137+140
[download] Destination: ...Beginners).f137.mp4
[download] Got error: 9849475 bytes read, 318544 more expected:
[download] Destination: ...Beginners).f140.m4a
[download] 100% of 14.00MiB in 00:01:58 at 121.43KiB/s
Finished, but some files could not be downloaded (see the errors above).
```

The folder had an unfinished `.f137.mp4` part video *without sound* and a `.f140.m4a` file *with only sound*. It looked as if the program had downloaded the video twice in two wrong formats.

7.2.2 Reading the evidence

- `137+140` means two formats: 137 is the 1080p video stream and 140 the AAC audio stream (Chapter 4). Downloading two files is *normal*; FFmpeg joins them at the very end. So the real question is: why was there no join?
- `Got error: ... more expected` means the connection closed before the video part was complete. The speed (121 KiB/s) shows a slow, unstable network.
- There is no `Retrying (1/10)` line. The download was abandoned at the first error, and without a complete video part FFmpeg had nothing to join.

7.2.3 Finding the cause

The `yt-dlp` command is known to retry 10 times. Why not here? The answer is in the library's source code, which pip installed in the environment's `site-packages/yt_dlp` folder:

```
# options.py -- only used by the yt-dlp terminal command
'--retries', dest='retries', default=10, ...

# downloader/http.py -- used by everyone, including our program
for retry in RetryManager(self.params.get('retries'), self.report_retry):
```

The default of 10 belongs to the *command-line parser*. When we use YoutubeDL from Python, `params.get('retries')` finds nothing and returns `None`, which means *zero* retries.

Important

A library used from Python does not always behave like the same tool used from the terminal. Defaults often live in the command-line layer. When a setting matters, write it explicitly in your options instead of relying on a default.

7.2.4 The fix and the test

One line in `build_options` (Chapter 5, step 6):

```
"retries": 10,
```

A new test protects it, so nobody removes it by accident:

```
def test_retries_after_network_drop(self):
    self.assertEqual(downloader.build_options(VIDEO, "/tmp/videos")["retries"],
                     10)
```

Then the real case was repeated. The same command continued the unfinished file (Resuming download at byte 49650475), downloaded the audio, and FFmpeg joined them. A check with ffprobe (installed together with FFmpeg) proved the result:

```
stream|codec_name=h264|codec_type=video|width=1920|height=1080|duration=906.64
stream|codec_name=aac|codec_type=audio|duration=906.69
```

Both streams are present and both last the full 15 minutes and 6 seconds.

7.3 Lesson 2: the terminal changed the link

7.3.1 The symptom

Look again at the link in the command above: `\?si\=`. The learner had pasted a normal link. The backslashes were added by *zsh*, the default shell on macOS and on many Linux systems. When you paste a URL, a *zsh* feature called *url-quote-magic* protects special characters such as `?` and `=` with a backslash, which would be correct *without* quotes. Inside quotes, however, the backslashes become part of the text, so Python received a broken link.

7.3.2 Why it matters

This video still worked by luck, because `yt-dlp` found the video id before the broken part. For a playlist link it would fail silently: `list\=` does not contain `list=`, so `is_playlist` returns `False` and the whole playlist would be saved without its folder and numbers.

7.3.3 The fix

A valid URL never contains a backslash, so it is always safe to remove them as soon as the link is read (Chapter 5, step 8):

```
url = url.replace("\\", "") # zsh may paste ? and = as \? and \=
```

The lesson: *clean user input at the place where it enters the program*. Then every function after it can trust the value.

7.4 Lesson 3: keep downloads out of the code and out of Git

In the first version, files without `-o` were saved right next to `downloader.py`. After a few downloads the code folder was full of large videos, mixed with the source files, and one careless `git add .` could have tried to upload gigabytes to GitHub.

Now the default is a separate folder:

```
SCRIPT_FOLDER = Path(__file__).resolve().parent
DOWNLOAD_FOLDER = SCRIPT_FOLDER / "downloads"
```

and `.gitignore` contains the line `code/downloads/`. You can ask Git to confirm that a file is ignored and by which rule:

```
git check-ignore -v "code/downloads/Me at the zoo.mp4"
```

```
.gitignore:222:code/downloads/ code/downloads/Me at the zoo.mp4
```

Two independent protections now exist: the folder rule and the older rules for media extensions such as `*.mp4`. Separating *program* files from *data* files is a good habit in every project.

7.5 Lesson 4: a new feature—only the sound

Language learners, podcast listeners, and music lovers often need only the audio. Downloading a full HD video for that wastes time and space: the lesson above is 144 MB as video but needs only 14 MB of audio.

The design keeps the program small:

- One switch on the command line: `-a / --audio`, with `action="store_true"` (Chapter 3).
- One new parameter, `audio=False`, passed from `main` through `download` to `build_options`. Its default keeps the old behaviour, so nothing changes for existing commands.
- One `if audio:` block that changes only what is different. The format becomes `bestaudio/best`, and an FFmpeg post-processor converts the result to MP3 (Chapter 4).
- When FFmpeg is missing, the post-processor is *not* added, because it would fail. The original audio file is kept instead. This is the same “work with what the computer has” idea as in `video_quality`.

Two tests cover both cases. They use `patch` to pretend that FFmpeg exists or does not exist, so they give the same result on every computer.

7.6 Lesson 5: stopping politely

Pressing `Ctrl+C` during a download used to print a long traceback that looked like a crash. Now `main` catches `KeyboardInterrupt` (Chapter 3) and prints one clear sentence:

```
Stopped. Run the same command again to continue.
```

The sentence is true because of how `yt-dlp` works: an unfinished download is kept as a `.part` file, and the next run continues it. Good messages tell the user *what happened* and *what to do next*.

7.7 Summary of the changes

Problem or need	Change, and how it is tested
A network drop stopped the video	<code>"retries": 10</code> ; a unit test and the real download repeated
zsh added backslashes to links	<code>url.replace(...)</code> in <code>main</code> ; tested with a real zsh-escaped link
Videos mixed with the code	<code>DOWNLOAD_FOLDER</code> and <code>code/downloads/</code> in <code>.gitignore</code> ; a unit test and <code>git check-ignore</code>
Only the sound	<code>--audio</code> and an MP3 post-processor; two unit tests and real single-video and playlist downloads
Ctrl+C looked like a crash	<code>try/except KeyboardInterrupt</code> ; tested by stopping a real download

Checkpoint

You can explain why a full HD download creates two files before the final `.mp4`, why the Python default for `retries` differs from the terminal command, and why the program removes backslashes from links.

Troubleshooting and Next Steps

Chapter goal

Fix the most common problems quickly and choose a small, useful improvement to build next.

8.1 Common problems

Message or symptom	What to do
<code>ModuleNotFoundError: No module named 'yt_dlp'</code>	The environment is not active, or the packages are not installed. Activate it and run <code>python -m pip install -r requirements.txt</code> .
Sign in to confirm you're not a bot	Open YouTube in a browser and sign in. Close the browser, then run the command again with <code>--browser firefox</code> (or your browser's name). Turning a VPN off or changing its server can also help.
No supported JavaScript runtime could be found	<code>deno</code> is missing. Activate the environment or run <code>python -m pip install deno</code> .
Got error: ... bytes read, ... more expected, and the folder has a <code>.f137.mp4.part</code> file and a <code>.f140.m4a</code> file	The network dropped while the video part was downloading. YouTube sends video and audio as two parts, which FFmpeg joins only when both are complete. The program already retries 10 times; if it still fails, run the <i>same command</i> again. <code>yt-dlp</code> continues the unfinished part, joins both, and deletes the temporary files.
Could not copy Chrome cookie database	Chrome is open and locks its cookies. Close every Chrome window and try again, or use another browser.
The file has no sound, or only low quality is saved	FFmpeg is not installed or not found. Install it and check <code>ffmpeg -version</code> in a new terminal.
It worked before but now fails with HTTP Error 403 or similar	YouTube changed something. Update the library: <code>python -m pip install -U "yt-dlp[default]"</code> .
Video unavailable or Private video	The video cannot be watched without permission. The other playlist videos are still downloaded and the program ends with the "some files could not be downloaded" message.
<code>zsh: no matches found</code> or the link is cut at <code>&</code>	Put the link inside quotes.
A whole playlist starts when you wanted one video	The link contains <code>&list=</code> . Remove that part and run again.

When you ask for help (for example in a GitHub issue), include your operating system, Python version, the exact command, and the complete error message. Remove any private information first.

8.2 Keep the project clean in Git

The project's `.gitignore` file tells Git which files never to save. It ignores the whole `code/downloads/` folder, media files anywhere (such as `*.mp4`, `*.mp3`, and `*.webm`), unfinished `*.part` files, and the `.venv` folder. Run `git status` before a commit: no video file should appear in the list.

8.3 Ideas for practice

Each idea needs only a few lines and practises one new skill. Add a test for each change.

1. **Choose the audio format.** Add an `--audio-format` option with `choices=["mp3", "m4a", "opus"]` and use its value as `preferredcodec` instead of the fixed `"mp3"`:

```
parser.add_argument("--audio-format", choices=["mp3", "m4a", "opus"],
                    default="mp3", help="file type for --audio")
```

2. **Maximum quality.** Add a `--max-height` option, for example `720`, to save disk space:

```
options["format"] = "bestvideo*[height<=720]+bestaudio/best[height<=720]"
```

3. **Several links at once.** Change `url` to `nargs="*"` and pass the whole list to `yd1.download`.
4. **Subtitles.** Save English and Persian subtitles next to each video:

```
options["writesubtitles"] = True
options["subtitleslangs"] = ["en", "fa"]
```

5. **Read links from a file.** Read one link per line from `links.txt`:

```
links = Path("links.txt").read_text().splitlines()
```

All available options are described in the [yt-dlp README](#). An option written on the command line, such as `--write-sub`, becomes a dictionary key in Python, such as `writesubtitles`.

Checkpoint

You can find the cause of a failed download from its message, update `yt-dlp`, and add a new option to the settings dictionary yourself.

Congratulations—you have built a complete, tested Python tool from an empty folder.

About the author

This tutorial and project were created by **Alireza Khajehvandi** to help new learners build something useful quickly and understand every line they run.

Continue learning and get in touch:

-  LinkedIn
-  YouTube channel
-  Telegram channel
-  Instagram
-  GitHub
-  Personal website and portfolio